Data-Driven Insights for Airbnb Markets in Europe

M.Sc. Business Intelligence and Process Management

Data Warehousing
Prof. Dr. Sebastian Fischer

WiSe 2025

Group 4:

Aurora Ruci

Karina Azizova

Nadi Kyaw

N. Itzel Hernandez

Table of Content

Table of Content **1**
1. Project Overview **2**
1.1 Context 2
1.2 Persona 2
1.3 Business Objective 2
1.4 Key analytical questions 3
1.5 Data & Scope Framing 3
1.6 End Product 3
2. Data Sources and Modeling **4**
2.1 Data Sources 4
2.2 Data Modeling 5
3. Technology Stack & Tools **6**
3.1 Local Storage 6
3.2 Google Cloud Platform as storage 6
3.3 Google Colab as data processing 7
3.4 Google Query as centralized data warehouse 7
3.5 Dbdiagram.io as designer 7
3.6 Tableau Cloud as business intelligence layer 7
4. The Pipeline **8**
4.1 Why ETLT Pipeline and its architecture 8
4.2 Extract to Transform 9
4.3 Load and Transform 13
4.4 From Gold to Tableau Cloud 15
5. Analysis & Insights **17**
6. Limitations and suggestions for future work **18**
References **20**

1. Project Overview

1.1 Context

In the European Union, short-term rental accommodation booked through major online platforms such as Airbnb, Booking and Expedia reached historically high demand during the summer of 2025. Between July and September, guests spent 398.1 million nights in short-stay rentals across the EU, marking an 8.7 % increase compared with the same quarter of 2024 and a 28 % growth compared with 2023, according to Eurostat (2026). This strong upward trend in guest nights highlights the growing importance of platform-based short-term rentals in European tourism and underscores the value of data-driven insights into pricing, demand and market selection.

1.2 Persona

A travel analytics platform that aggregates Airbnb listing data (pricing, location, reviews, seasonality, events) to provide insights and personalized recommendations to travelers. The project is specifically being delivered to a Market Research analyst responsible for exploring short-term rental market trends across major European cities. The stakeholder has an intermediate level of data expertise and is comfortable using dashboards and aggregated metrics, but has no experience interacting with raw data. The analyst relies on static reports and pre-aggregated summaries that limits flexibility and cross-dimensional exploration.

1.3 Business Objective

The company's business goal is to enhance the quality and relevance of its market and listing recommendations by leveraging data-driven insights from Airbnb. By improving recommendation accuracy, the company aims to ensure that suggested listings and markets consistently align with observed indicators of high value, such as pricing dynamics, demand patterns, listing quality, and external factors, such as events happening in the city.

1.4 Key analytical questions

Question 1: How can Airbnb market data be used to identify high-value listings and optimal booking periods across major European cities?

Question 2: How do Airbnb prices differ between weekdays and weekends, and how do seasonal trends influence price levels across cities and room types?

Question 3: Which cities show the highest demand for Airbnb listings, and how does demand relate to average pricing levels?

Question 4: Do we have a sufficient and available density of active listings within a 2-kilometer radius of major event venues (with a capacity of > 50,000 visitors)?

Question 5: How does guest sentiment vary across listings, and how is overall sentiment related to perceived listing quality and review scores?

1.5 Data & Scope Framing

This project explores short-term rental listings across 14 major European cities using publicly available Airbnb-related data. The dataset includes listing information such as location, price per night, availability, room type, amenities, number of bedrooms, guest reviews and ratings, and host characteristics like superhost status. Pricing data covers both weekday and weekend rates, allowing basic comparisons of price patterns over time. Spatial information and external data on major events are included to support the identification of pricing differences across different cities through time. The analysis is exploratory and descriptive, and does not include predictive modeling or real-time decision-making.

1.6 End Product

The end product of this project is an interactive, exploratory analytics dashboard designed for market research analysts working with short-term rental data. The dashboard allows users to filter and compare European cities in order to identify where high-value and convenient listings are most commonly located. Through interactive visualizations, the user can explore patterns related to pricing, availability, listing quality, and seasonality across markets. By consolidating these insights into

an exploratory environment, the dashboard supports faster recommendation development and helps uncover patterns that contribute to more accurate and data-driven recommendations for clients.

2. Data Sources and Modeling

2.1 Data Sources

Open sources were used for the purpose of this project, which consists of:

- **Listings** from insideariBnb.com in the form of comma separated values or csv formats, and including location and neighbourhood details, property and room characteristics, capacity, amenities, and pricing. It also includes availability indicators, stay restrictions, guest reviews and ratings, and estimated performance metrics such as occupancy and revenue. In addition, host-level attributes such as experience, responsiveness, and superhost status are included to support analysis of factors associated with high-value listings;

- **Reviews** from insideariBnb.com in the form of comma separated values or csv formats, consisting of extended textual feedback provided by guests after their stay. This information was used mainly as a basis for the sentimental analysis;

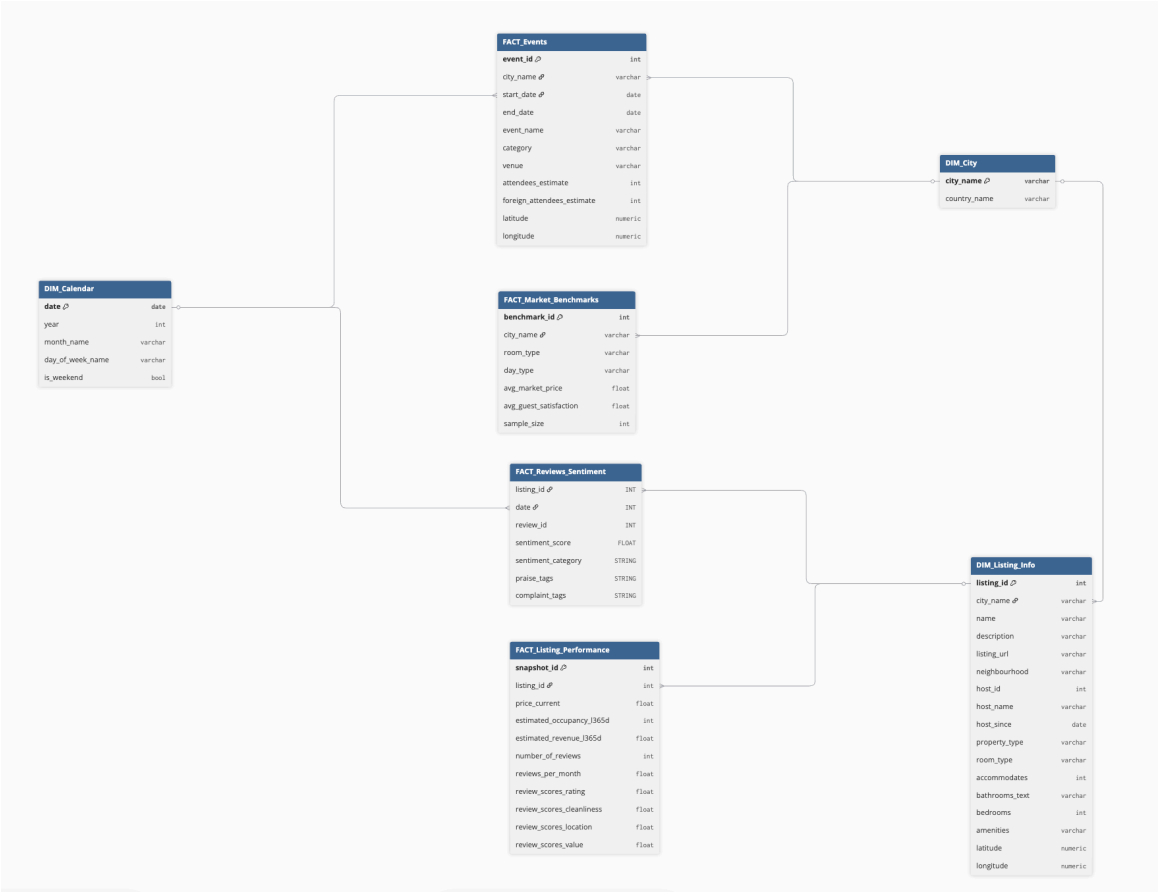
- **Pricing** from kaggle.com in the form of comma separated values or csv formats, and capturing weekday and weekend Airbnb prices, along with key attributes such as room type, number of bedrooms, guest satisfaction, host characteristics, and spatial indicators like distance from the city centre across major European cities. Pricing is added to support comparative analysis of pricing patterns;

- **Events** data is AI-collected real-world data from different open sources. The AI acted as a sophisticated research aggregator, scanning and unifying fragmented schedules from global festival calendars, conference registries, and sports federations into a single, structured resource.

2.2 Data Modeling

Approach

The data model was designed to support exploratory and comparative analysis of pricing and demand patterns of the listings across the different cities, rental types and locations. Given the heterogeneous sources needed to answer the different analytical questions, as well as the distinct grain and measures of them, the most appropriate structure was a Galaxy schema. This schema enabled a flexible analysis through shared dimensions (city, calendar, listing information) while supporting the exploration of pricing, demand, quality and external factors.



Grain definition

Fact Tables

The data model is formed by multiple fact tables, each defined at a different level of granularity to fulfill the analytical concerns of the project.

- *FACT_Listing_Performance:* Each row represents the state and performance;

- *FACT_Reviews_Sentiment:* Each row represents a single guest review for a listing on a specific date;
- *FACT_Market_Benchmarks:* Each row represents an aggregated market benchmark for a specific city and listing type under a given condition;
- *FACT_Events:* Each row represents a single event occurring in a specific city over a defined time period.

Dimension Tables

The fact tables are connected through a set of conformed dimensions that enable consisting filtering and analysis across the model:

- *DIM_City:* reflects geographic context;
- *DIM_Listing_Info:* descriptive attributes of listings where applicable;
- *DIM_Calendar:* temporal context supporting time-based analysis.

3. Technology Stack & Tools

3.1 Local Storage

We used our local storage for downloading the datasets from InsideAirbnb and then transferred them to Google Cloud Platform.

3.2 Google Cloud Platform as storage

We chose to use Google Cloud Platform (GCP) as the core infrastructure for this project's data warehousing architecture, creating a real-world cloud environment for storing and analyzing data. Fortunately, we could use GCP without charging any costs because it offered a free trial with $300 welcome credits, which was quite good for us. It played as the foundation for the Bronze, Silver, and Gold Data Layers, acting as a central data storage and connecting data processing tools like Google Colab and BigQuery and visualization tools like Tableau. In short, GCP provided the infrastructure needed to build an end-to-end pipeline that was consistent with the modern data architecture.

3.3 Google Colab as data processing

We used Google Colab mostly for loading, cleaning, creating, and directly saving on GCP. It basically changed unstructured data into structured datasets, which were suitable for the analysis steps. It also linked raw data ingestion and the analytical warehouse. The choice of Google Colab was also driven by educational considerations. Prior to the project, the team had already begun practicing Python as part of a Data Science course and aimed to further develop these skills within the project. Using Google Colab provided a familiar environment for applying Python-based data processing while supporting the team's learning objectives.

3.4 Google Query as centralized data warehouse

Google BigQuery played as the project's analytical data warehouse and represents the gold layer, where data is stored after it has been cleaned in Google Colab. It was efficiently storing the large Airbnb datasets, linking tables and doing aggregations. Besides, it also gave a reliable source of data for Tableau Dashboards and followed best practices for data warehousing, where heavy processing could be done in the warehouse instead of doing it on the visualization tool.

3.5 Dbdiagram.io as designer

We used dbdiagram.io to design our project's Galaxy Schema. It was helpful in visualizing the relationships between datasets, clearly defining primary and foreign keys.

3.6 Tableau Cloud as business intelligence layer

Last but not least, we decided to use Tableau Cloud as the final visualization and presentation layer of the project, providing a central tool for users to interact with data and grow insights. It is a good platform for data preparation, which is fast, flexible, collaborative, and focused on publishing and sharing analytics content. After we connected directly to BigQuery to create interactive dashboards, we joined our datasets, such as listings, price, events, and reviews based on Galaxy Schema we created in dbdiagram.io. The end product of the project was to transform the

analyzed data into practical insights, helping to involve a company and client perspective, not just a technical one.

4. The Pipeline

4.1 Why ETLT Pipeline and its architecture

The project team adopted an ETLT (Extract-Transform-Load-Transform) approach as a pragmatic and flexible solution that reflected both the educational context of the project and the uncertainty present at its early stages.

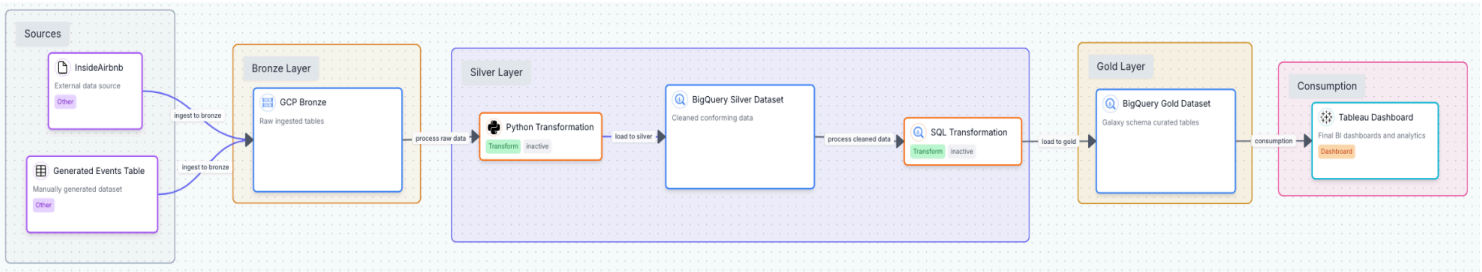
One of the key reasons for this choice was the team's strong motivation to engage with modern data engineering practices while simultaneously reinforcing previously acquired technical skills. At the time of initiating the project, the team had already started a coursework in Data Science, with a particular focus on Python-based data processing, while structured SQL-based data transformations were introduced later within the scope of the current discipline. As a result, Google Colab was selected as an initial processing environment to practice and apply Python skills in a realistic data engineering context, especially during the early stages of data exploration and preparation.

In addition, during the initial phases of the project, the team faced challenges in structuring both the toolset and the overall data processing workflow. The final set of data sources, as well as their formats and quality, were not fully known in advance. Under these conditions, committing early to a strictly defined transformation logic or a rigid pipeline architecture would have limited the team's ability to adapt. The ETLT approach provided the necessary flexibility to perform preliminary transformations externally, experiment with data structures, and gradually converge toward a more stable and structured processing model.

Furthermore, the ETLT architecture supported an exploratory and iterative working style that was well suited to the project's evolving analytical goals. Early transformations enabled schema-on-read-like behavior and facilitated exploratory data analysis, while the final transformation stage allowed the team to apply structured, analytics-oriented logic once the data model and business requirements became clearer.

Overall, the choice of the ETLT approach was driven by educational objectives, initial uncertainty regarding data availability and processing structure, and the need for a flexible framework that could evolve alongside the team's technical maturity and project understanding.

The pipeline's architecture is as follows:



4.2 Extract to Transform

From Data sources to GCP Buckets

The code[1] (Notebook: 01_Listings and 01_Reviews) searches the given url which contains cities all around the world, finds the cities defined in TARGET_LOCATIONS, and saves them temporarily within Google Colaboratory.

```
# --- 1. CONFIGURATION: EDIT THIS SECTION ---
DATA_PAGE_URL = "http://insideairbnb.com/get-the-data.html"
BRONZE_LOCAL_DIR = "BRONZE_RAW_CHECKPOINT" # Temporary folder in Colab
TARGET_LOCATIONS = {
    "Amsterdam", "Athens", "Barcelona", "Berlin", "Bristol",
    "Brussels", "Budapest", "Dublin", "London", "Lyon",
    "Munich", "Oslo", "Paris", "The Hague", "Venice"
}
TARGET_FILE_TYPE = "listings.csv.gz"
# --- End of Configuration ---
```

The files are named based on the city used to find the file in the previous code. This assured that we could later in the transformation steps add a column with the city for each listings file. This way we could have a clearer separation between cities once we join the listings in one dataset.

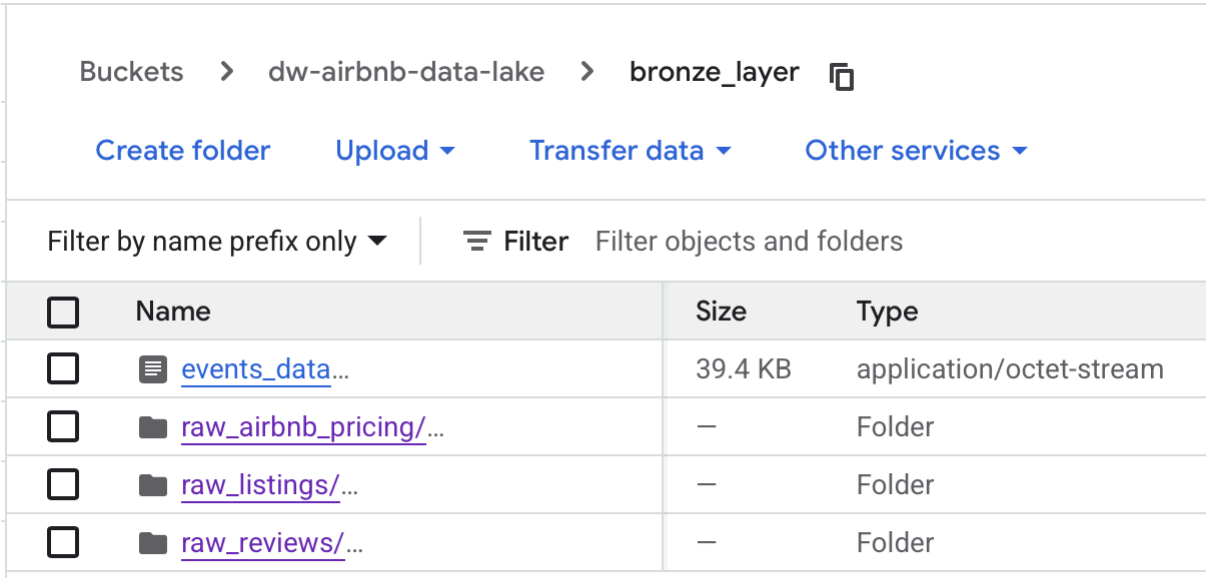
[1] Python code used for the Google Colab notebooks hereafter was written with the help of Google's Gemini AI and reviewed for completeness.

```python

def download_and_process_file(href, location_name):
    """Downloads the GZ file, unzips it, and saves it locally as CSV."""
    clean_name = re.sub(r'[\\/*?:"<>|]', "", location_name)
    local_filename_csv = os.path.join(BRONZE_LOCAL_DIR, f"{clean_name}_listings.csv")

```

Once downloaded, the data is added to the relevant GCP buckets. At this stage, the 61 files serve as our bronze layer.



The first part of the Transformation is carried out with Python through Google Colab, directly transforming the files in the **dw-airbnb-data-lake/bronze_layer** bucket. As mentioned earlier, the early choice to carry out this process this way was part of the team's initiative to practice our code reading and understanding skills. We performed the basic, initial cleaning and added the cleaned datasets to a new GCP bucket as a parquet file. The parquet files were then subsequently connected to BigQuery as External tables into its own "silver" dataset. The External tables allowed for live connection between GCP Storage and BigQuery, assuring the pipeline could be rerun with updated city listings, pricing, or reviews data. We outline below the transformations taken on each dataset.

We take Listings (Notebook: 02_Listings_cleaning) as a primary example, as it was the first dataset the team processed and it served as example for the datasets that followed (excl. Events). As the original table had more columns than needed, we first decided to only keep relevant columns for our analysis. Second, as mentioned earlier, we wanted to have a clear separation between cities when we would join the

data together for visualisation and analysis. The name of the city is taken from the file name as **city_name** (T1) and added to a new column as **city** (T3).

```
# --- T1: Add 'city' column ---
clean_filename = file_name.replace(SOURCE_PREFIX, '')
city_name = clean_filename.split('_')[0]

gcs_path = f'gs://{BUCKET_NAME}/{file_name}'
```

```
# --- T3: Add City ---
df.insert(1, 'city', city_name)
```

Next, during initial exploratory analysis, we realized non-Euro countries showed the price of a listing per night in local currency. This was greatly affecting the test visualization, as Budapest, for example, was significantly more expensive than other cities because of the difference between Euro and Hungarian Forint. To scale the data, we approximated the exchange rate as of November 2025 and transformed the Price column for non-Euro countries. The price was first cleaned as it was originally a string, by removing non-numerical values. Finally, the currency rate was multiplied with the price if the **city_name** column matched the city in the pre-defined list **CURRENCY_RATES.**

```
# --- T4: Price Cleaning & CURRENCY CONVERSION ---
if 'price' in df.columns:
    # 1. Clean the string (remove $ and ,)
    df['price'] = df['price'].astype(str).str.replace(r'[^0-9.]', '', regex=True)
    # 2. Convert to float
    df['price'] = pd.to_numeric(df['price'], errors='coerce')

    # 3. Apply Currency Conversion if needed
    if city_name in CURRENCY_RATES:
        rate = CURRENCY_RATES[city_name]
        df['price'] = df['price'] * rate
        print(f"  💱 Converted {city_name} prices (Rate: {rate})")
```

Following a check for NaN values, we concluded that out of the relevant columns for analysis, the pricing column was highly important to take care of. We decided to not drop the values, but instead add a column **listing_status** which would be populated with “Unavailable” if the price column is a NaN value, “Available” if not. This way, we keep the original listing for certain analytical questions where price is not relevant,

with the option to ignore it when price is necessary information. Other important columns had no NaN values, which simplified the decision making process on how to deal with NaN values by a great margin.

The last step of this initial transformation was data type cleaning. More specifically, we converted date columns to datetime objects, ID columns to Int64, and Count columns to Int64. We turn these numerical data to Nullable integers because This helps avoid unintended floating-point math and allows us to keep integers and missing values.

```python
# 1. Convert Date columns to datetime objects
date_cols = ['last_scraped', 'host_since']
for col in date_cols:
    if col in main_df.columns:
        main_df[col] = pd.to_datetime(main_df[col], errors='coerce')

# 2. Convert ID columns to 'Int64' (Nullable Integer)
id_cols = ['id', 'host_id']
for col in id_cols:
    if col in main_df.columns:
        main_df[col] = pd.to_numeric(main_df[col], errors='coerce').astype('Int64')

# 3. Convert Count columns to 'Int64' (Nullable Integer)
count_cols = [
    'bedrooms', 'minimum_nights', 'maximum_nights', 'availability_30',
    'availability_60', 'availability_90', 'availability_365',
    'number_of_reviews', 'estimated_occupancy_l365d'
]
for col in count_cols:
    if col in main_df.columns:
        main_df[col] = pd.to_numeric(main_df[col], errors='coerce').astype('Int64')

print("--- New Data Types ---")
main_df.info()
```

For the rest of the datasets, this transformation code was kept in mind and adapted for the specific cases. We outline below differences in transformation changes or additional changes carried out for them. Additionally to the city name, to the Pricing dataset was added a **day_type** column to differentiate between weekdays and weekends.

```python
# Extract 'weekdays' or 'weekends' from filename and add as 'day_type' column
day_type_match = re.search(r'_(weekdays|weekends)\.csv$', filename, re.IGNORECASE)
if day_type_match:
    df['day_type'] = day_type_match.group(1).capitalize() # Capitalize for consistency
else:
    df['day_type'] = 'Unknown' # Default if day type cannot be extracted
```

Additionally, the pricing data was used for some Exploratory Data Analysis to compute useful data, such as average price per city/per day type and real sums.

Regarding the Reviews data, it followed the same transformation steps as the Listings data relevant to its own original columns. A sentiment analysis was conducted on the Reviews dataset (Notebook: 02_Reviews_SentimentAnalysis) using the python package VaderSentiment **(SentimentIntensityAnalyzer)** to add a sentiment score for each listing to be used in the final visualization. Finally for the Review dataset, keywords were extracted for complaints/praises.

The Events dataset was generated without the need to pre-transform before uploading it to BigQuery.

4.3 Load and Transform

The load-phase from our GCP buckets took place in Big Query. The previously saved parquet files were then connected to BigQuery via external tables (to keep the link to the source files). They were saved as tables under the “silver” dataset. The schema was automatically detected by BigQuery and double checked after connection was successful.

The second Transformation stage of the pipeline was executed directly within BigQuery using SQL, transitioning data from the "silver" dataset to the final "gold" layer. This required redefining tables according to Galaxy Schema (see 2.2 Data Modeling). The architectural choice to implement a Galaxy Schema was driven by the specific requirements of our visualization layer in Tableau, particularly the need for independent map layers without data duplication. Unlike the previous step, these transformations resulted in native BigQuery tables rather than external connections, ensuring optimized query performance for the dashboard.

We outline below the architectural decisions and specific transformations applied to the data.

To address the complexity of analyzing distinct geospatial entities - specifically the relationship between short-term rental listings and local events - we moved away from a single flat table approach. Instead, we constructed a model relying on DIM_City, DIM_Calendar, DIM_Listing_Info shared across multiple independent

FACT_Events, FACT_Listing_Performance, FACT_Reviews_Sentiment, FACT_Market_Benchmarks.

To facilitate continuous time-series analysis and prevent sparse data gaps during visualization, the **DIM_Calendar** table was programmatically generated rather than derived from transactional data. The query utilizes a Common Table Expression (CTE) named date_range which leverages BigQuery's GENERATE_DATE_ARRAY function. This function constructs a dense array of dates incrementing by one day between '2025-01-01' and '2026-12-31'. The UNNEST operator flattens this array into a tabular format. The final SELECT statement enriches this core timeline by deriving user-friendly attributes:

- Functions such as EXTRACT(YEAR...) and FORMAT_DATE('%B'...) allow for intuitive slicing by Month and Year in Tableau;
- A CASE statement identifies weekends (DAYOFWEEK 1 or 7). This boolean flag (is_weekend) is critical for analyzing price elasticity, enabling the business to differentiate between leisure and business travel patterns.

The construction of **DIM_City** required a sophisticated approach to ensure accurate mapping. Early iterations used a simple DISTINCT selection of city names, but this proved insufficient for global visualization due to geocoding duplications (e.g., 'Berlin' existing in both Germany and USA).The final implementation employs a UNION ALL strategy to aggregate location data from three distinct sources: listings, pricing, and events. This ensures the dimension is exhaustive. Crucially, the query calculates Geospatial Centroids for each city using the AVG(latitude) and AVG(longitude) aggregate functions grouped by city_name. By mathematically averaging the coordinates of all actual data points within our dataset, we force the visualization engine to center the map on the relevant European locations, effectively overriding default and incorrect geocoding behaviors.

The **DIM_Listing_Info** table serves as the central descriptive node of the Galaxy Schema. The primary technical challenge addressed here was Timestamp Normalization. The source data contained timestamps in varying precisions (seconds, microseconds, and nanoseconds). The SQL logic implements a robust CASE statement to dynamically detect the scale of the integer:

- values exceeding 10^16 are identified as nanoseconds and divided by 10^9
- values exceeding 10^13 are identified as microseconds and divided by 10^6.
- values exceeding 10^10 are identified as milliseconds and divided by 10^3.

This normalized integer is then cast to TIMESTAMP_SECONDS and converted to a DATE object, ensuring the host_since field is consistent across all records regardless of the source system's logging format.

FACT_Events follows the same rigorous timestamp normalization logic applied to DIM_Listing_Info for its Start_Date and End_Date fields. Critically, this table retains venue_latitude and venue_longitude as distinct measures. This separation from the city centroid allows for the creation of multi-layered geospatial visualizations, enabling the analysis of proximity effects between specific event venues and surrounding accommodation.

FACT_Listing_Performance acts as a slim, high-performance fact table. It strips away text-heavy columns, retaining only the listing ID (foreign key), coordinates, and quantitative metrics such as price_current, estimated_revenue, and review_scores. This vertical partitioning optimizes query performance for aggregation-heavy visualizations.

Unlike the direct row-level transformations of other tables, **FACT_Market_Benchmarks** creates a derived analytical layer. It aggregates the raw pricing dataset using a GROUP BY clause on city, room_type, and day_type. By calculating weighted averages (AVG(realSum)) at this stage, we provide a pre-computed baseline for market analysis. This allows the dashboard to instantly compare a specific listing's price against the “market average” for that specific room type and day of the week without performing expensive calculations on the fly.

The **FACT_Review_Sentiment** table represents the final enrichment layer. The major changes took place in the first Transformation process.

4.4 From Gold to Tableau Cloud

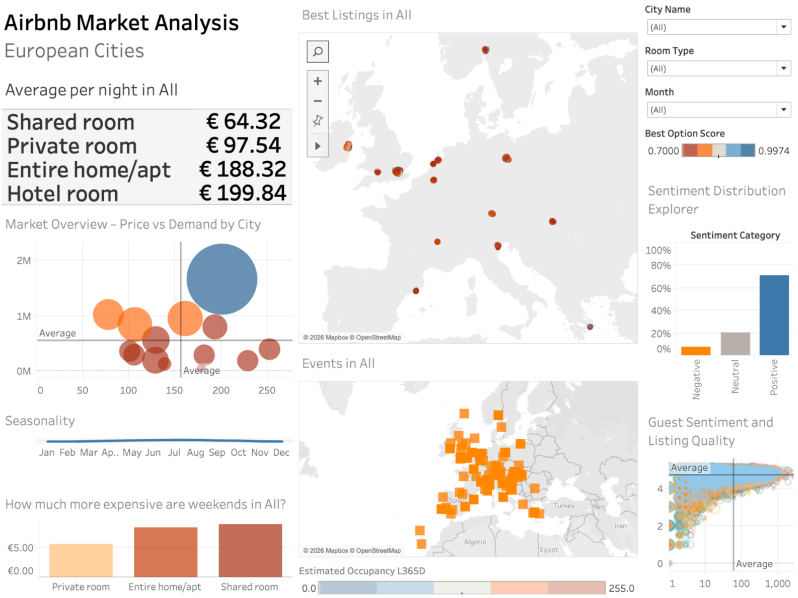
The final phase of the project pipeline involved establishing a seamless integration between the data warehouse and the visualization interface. We implemented a Live

Connection between Google BigQuery and Tableau Cloud, bypassing the use of static extracts.

Upon ingestion, the Galaxy Schema was reconstructed within Tableau's logical data layer using Relationships. Unlike traditional physical joins, this model allowed the distinct fact tables (FACT_Events, FACT_Listing_Performance, FACT_Reviews_Sentiment, FACT_Market_Benchmarks) to remain independent until query time, preventing data duplication and preserving the granularity of the 'Conformed Dimensions'.

The selected choice for the visualization was a Dashboard in order to meet the requirements of the Persona, which aligns with having an interactive object where they can analyze trends across short-term rents in Europe. In the sense that the analyst can explore through different dimensions the most important trends (accommodation type, time, location, events).

Visualization



This dashboard was designed to analyze Airbnb market dynamics in an exploratory analysis across major European cities by integrating pricing benchmarks, demand indicators, spatial data, seasonal trends, event information, and guest sentiment into a unified analytical view.

The purpose is to identify high-value markets and listings, understand price behavior over time and between weekdays and weekends, evaluate demand concentration, assess listing availability around major events, and relate guest sentiment to perceived listing quality. Visualization choices were designed to follow best practices, using appropriate chart types for each analytical task, clear baselines and reference lines, and a structured visual hierarchy that emphasizes key insights. Interactive filters allow the user to focus on specific cities or room types without changing the underlying logic, helping the dashboard present insights clearly and consistently.

Color is used consistently across the dashboard, with neutral tones as the base and accent colors applied only where they add meaning, such as highlighting price levels, sentiment categories, or important data points. This approach helps reduce visual clutter and makes it easier to compare information across different views. The color palette was selected to remain interpretable for color-blind users by avoiding red-green contrasts and ensuring that key information is not conveyed through color alone.

5. Analysis & Insights

The analysis shows that Airbnb prices and demand vary clearly across cities, with higher prices often linked to popular and central locations. Weekend stays are generally more expensive than weekdays across all room types, and seasonal patterns reveal times of the year when prices are lower. High-performing listings tend to cluster in central areas and near major event venues, highlighting the importance of location. Guest reviews are mostly positive, and better-rated listings are closely associated with higher perceived quality.

Berlin: Applied Dashboard Insights

When Berlin is selected in the dashboard, the analysis shows how the analytical questions are answered. The *Best Listings in Berlin* map highlights that high-performing listings are mainly located in central areas of the city. These listings achieve higher *Best Option Scores*, due to a combination of the most convenient prices plus strong ratings.

The *Average per night in Berlin* panel shows clear price differences by room type. Shared rooms are the most affordable option, followed by private rooms, while entire homes or apartments are significantly more expensive. Hotel rooms show the highest average prices.

The comparison between weekdays and weekends indicates that prices increase across all room types during weekends. This increase is strongest for entire homes and apartments, smaller for private rooms, and lowest for shared rooms. The seasonality chart shows that prices tend to rise during the middle of the year and fall during winter months, making off-peak periods more affordable for travelers.

In terms of demand, the *Price vs Demand* view shows that Berlin has relatively high demand while remaining close to the average price level. This suggests that Berlin offers a strong balance between popularity and affordability compared to other European cities.

The events view highlights major events such as the Berlin Marathon. Around these events, nearby listings show higher estimated occupancy levels, indicating increased short-term demand.

Finally, the sentiment analysis shows that most guest reviews in Berlin are positive. Listings with higher review scores and more reviews tend to have more positive sentiment, confirming the relationship between listing quality and guest satisfaction. Overall, the Berlin example demonstrates how the dashboard combines pricing, demand, seasonality, location, events, and sentiment to identify high-value listings and better booking periods.

6. Limitations and suggestions for future work

This section outlines the limitations identified during the project's execution and reflects on the technical constraints encountered. The initial discrepancy between the team's technical proficiency and the architectural complexity made the translation of theoretical concepts into a fully automated production environment challenging. The team had a mostly business background and while it had its limitations, it proved to be a good ground for learning and development. The learning process throughout

the semester has made it possible for the team to identify parts of the project that could be improved in the future.

The primary limitation of the current architecture is the absence of a fully automated, dynamic ETLT pipeline. Currently, ensuring data freshness in the visualization layer requires manual intervention to execute the extraction scripts, load data to Google Cloud Platform, and trigger the SQL transformation queries. This reliance on manual execution creates a significant operational bottleneck and prevents the system from acting as a "real-time" monitoring tool. Nonetheless, the nature of our data does not have the goal to act as a “real-time” monitoring tool, as the data itself is scraped from Airbnb in infrequent time intervals. Future development would prioritize implementing an orchestration layer to automate these steps simply for convenience’s sake.

References

Beratungscontor. (205). *Google Cloud Platform (GCP)*. https://beratungscontor.de/lexikon/google-cloud-platform-gcp/

DataCamp. (2025). *Introduction to Tableau*. https://www.datacamp.com/courses/introduction-to-tableau

DataCamp. (2025). *Intermediate SQL*. https://www.datacamp.com/courses/intermediate-sql

Eurostat. (2026, January 8). *Short-stay accommodation statistics*. https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20260108-1

Inside Airbnb. (2025). *Get the data*. https://insideairbnb.com/get-the-data/

Kaggle. (n.d.). *Airbnb datasets*. https://www.kaggle.com/datasets/thedevastator/airbnb-prices-in-european-cities

RisingWave. (2024.). *Schema-on-write vs. schema-on-read*. https://risingwave.com/blog/data-management-schema-on-write-vs-schema-on-read/

Zhang, Y., & others. (2021). *[Article on data analytics and decision-making]*. *Journal of Retailing and Consumer Services, 61*, 102533. https://www.sciencedirect.com/science/article/pii/S0261517721000388